NETWORKING

gRPC
Visual Guide

Contract-first RPC —
protobuf, HTTP/2, and four calling patterns.

ANIMATED WALKTHROUGH
Step through pages to play the animation

STEP 1/5

Protobuf: The IDL

Define the contract once — generate clients and servers in any language.

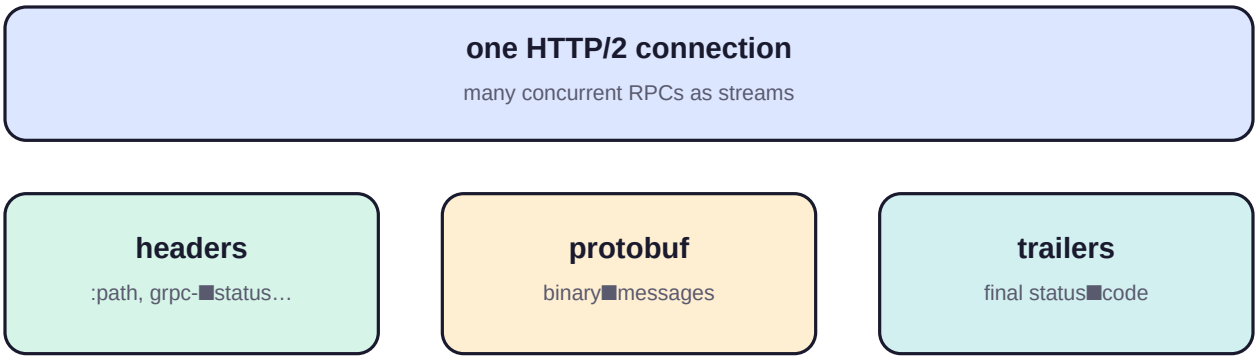
```
service Greeter {
  rpc SayHello (HelloReq) returns (HelloReply);
}

message HelloReq { string name = 1; }   // binary, typed, versioned
```

- Binary serialization: far smaller and faster than JSON — field numbers, not names, on the wire.
- Backward compatible: add fields (never reuse numbers); old clients ignore what they don't know.
- Codegen: protoc emits typed stubs for 10+ languages — the contract IS the documentation.

STEP 2/5

HTTP/2 Under the Hood

gRPC rides multiplexed HTTP/2 streams — not websockets, not custom TCP.



- Trailers carry the final gRPC status code — the RPC isn't done until trailers arrive.
- Multiplexing: hundreds of RPCs share one connection — no head-of-line blocking between calls.
- TLS is standard; plaintext (h2c) exists but is rare outside the datacenter.

STEP 3/5

Four Calling Patterns

Streaming in either or both directions.

Unary

one request → one response. The classic RPC.

Server streaming

one request → stream of responses. Live feeds, search results.

Client streaming

stream of requests → one response. Uploads, telemetry ingestion.

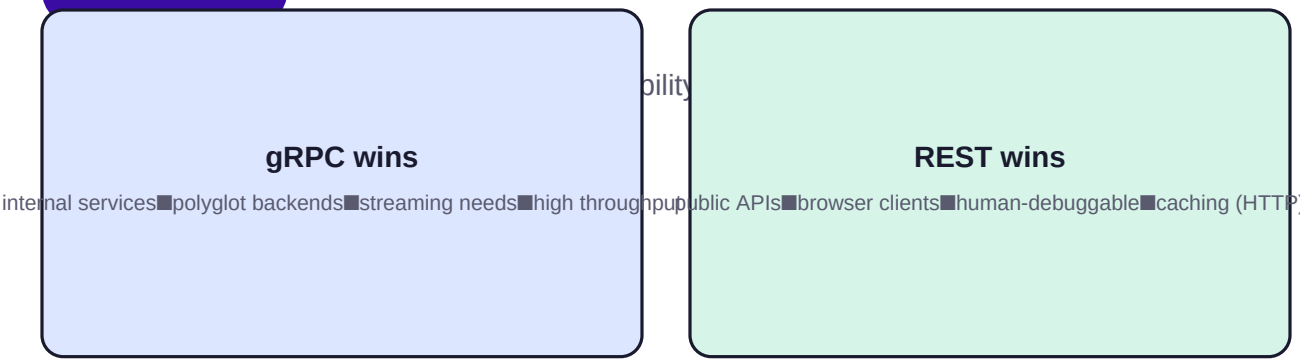
Bidirectional

both sides stream independently. Chat, real-time collaboration.

Backpressure is built in: HTTP/2 flow control paces fast producers against slow consumers.

STEP 4/5

gRPC vs REST



- gRPC payloads are binary — curl can't read them; grpcurl and proxies fill the gap.
- Browsers can't speak gRPC natively — grpc-web needs a translating proxy.
- Many teams run both: gRPC inside, REST/JSON at the edge.

STEP 5/5

Production Realities

What running gRPC at scale teaches you.

- Load balancing: one long-lived HTTP/2 connection defeats round-robin LBs — use client-side LB or proxies (Envoy).
- Deadlines propagate: set per-RPC timeouts and pass them down the call chain — don't let one slow call hang the mesh.
- Interceptors = middleware: auth, logging, retries, and metrics hook in uniformly.
- Health checking + reflection: standard probes keep orchestrators happy; reflection aids debugging.

Retries need idempotency: safe to retry unary reads; think twice before retrying non-idempotent writes.

STEP ✓I✓

Key Takeaways

gRPC — Remember

- Protobuf IDL: binary, typed, versioned contracts with generated stubs.
- Transport is HTTP/2: multiplexed streams, trailers carry the status code.
- Four patterns: unary, server-streaming, client-streaming, bidirectional.
- gRPC for internal polyglot services; REST for public APIs and browsers.
- Mind L7 load balancing, deadline propagation, and retry idempotency.

Keep exploring

Pair this guide with the matching video walkthroughs
and hands-on labs at interstitiumlabs.dev